Housing Market API Documentation

Author: *Jonathan Owens*

Module: COMP3011

Date: *13/03/2026*

---

1. API Overview

The Housing Market API provides access to UK property transaction data and analytical insights derived from historical housing sales.

The API allows users to:

- Search property transactions by location and price
- Find average price per area
- Filter results by property type and date range
- Analyse price trends by year
- Identify the most expensive streets in a given area
- Create, Read, Update and Delete property transactions

The API is implemented using FastAPI and returns responses in **JSON format**.

---

2. Access the API

Local development: http://127.0.0.1:8000

Local frontend: http://127.0.0.1:8000/app

Deployed version: https://<your-deployment-url>

The system consists of a REST API built using FastAPI and a browser-based frontend interface implemented using HTML, CSS, and JavaScript.

The frontend communicates with the API via HTTP requests to retrieve property data and analytical insights. Responses are returned in JSON format and visualised in the frontend using charts and tables.

---

3. Authentication

This API **does not require authentication**.

All endpoints are publicly
accessible.

---

4. Endpoints

4.1 Property Search

Returns property transactions filtered by location, property type, and price range.

Endpoint

GET /properties

Query Parameters

Parameter	Type	Description
location	string	Town, city, or postcode
property_type	string	D (Detached), S (Semi), T (Terraced), F (Flat)
min_price	integer	Minimum property price
max_price	integer	Maximum property price
start	date	Start date for sale
end	date	End date for sale
sort_by	string	Sorting option (price, -price, date, -date)
limit	integer	Maximum number of records returned
offset	integer	Number of records to skip

Example Request

GET /properties?location=London&property_type=F&limit=10

Example Response


```
[
    {
        "id": 10231,
        "price": 450000,
```

```
        "transfer_date": "2021-06-15",
        "postcode": "SW1A 1AA",
        "street": "Baker Street",
        "town_city": "London",
        "property_type": "F",
        "tenure": "F"
    }
]
```

4.2 Average Price Endpoint

Returns the average property price for a given location.

Endpoint

GET /properties/average-price

Parameters

Parameter	Type	Description
location	string	Town or city

Example Request

GET /properties/average-price?location=London

Example Response


```
{
    "average_price": 520000
}
```

4.3 Count Endpoint

Returns the number of properties in the Database.

Endpoint

GET /properties/count

Parameters

It does not take any paramters

Example Request

GET /properties/count

Example Response

```
{
    "total_properties" = 5798901
}
```

4.4 Get Property by ID

Retrieves a single property record using its unique identifier.

Endpoint

GET /properties/{property_id}

Path Parameters

Parameter	Type	Description
property_id	integer	Unique identifier of the property

Example Request

GET /properties/10231

Example Response

```
[
    {
        "id": 10231,
        "price": 450000,
        "transfer_date": "2021-06-15",
        "postcode": "SW1A 1AA",
        "street": "Baker Street",
        "town_city": "London",
        "property_type": "F",
        "tenure": "F"
    }
]
```

4.5 Create Property

Creates a new property record in the database.

Endpoint

POST /properties

Request Body

JSON object containing property information.

Field	Type	Description
price	integer	Sale price
transfer_date	date	Date of property transfer
postcode	string	Property postcode
street	string	Street name
town_city	string	Town or city
property_type	string	Property type
tenure	string	Tenure type

Example Request

```
{
    "price": 550000,
    "transfer_date": "2024-03-12",
    "postcode": "M1 2AB",
    "street": "Oxford Road",
    "town_city": "Manchester",
    "property_type": "F",
    "tenure": "L"
}
```

Example Response

```
{
   "id": 12001,
    "price": 550000,
    "transfer_date": "2024-03-12",
    "postcode": "M1 2AB",
    "street": "Oxford Road",
    "town_city": "Manchester",
    "property_type": "F",
    "tenure": "L"
}
```

Status code: 201 Created

4.6 Update Property

Updates an existing property record.

Endpoint

PUT /properties/{property_id}

Path Parameters

Parameter	Type	Description
property_id	integer	Property identifier

Example Request

PUT /properties/12001

Request body:


```
{
  "price": 575000
}
```


Example Response


```
{
  "id": 12001,
   "price": 575000,
   "transfer_date": "2024-03-12",
   "postcode": "M1 2AB",
   "street": "Oxford Road",
   "town_city": "Manchester",
   "property_type": "F",
   "tenure": "L"
}
```

4.7 Delete Property

Deletes a property record from the database.

Endpoint

DELETE /properties/{property_id}

Path Parameters

Parameter	Type	Description
property_id	integer	Property identifier

Example Request

DELETE /properties/12001

Response

204 No Content

The property is successfully removed from the database.

4.8 Price Trend Analytics

Returns average property prices grouped by year.

Endpoint

GET /analytics/price-trend

Parameters

Parameter	Type	Description
location	string	Town or city
property_type	string	D, S, T, F

Example Request

GET /analytics/price-trend?location=London&property_type=F

Example Response


```
[
  {
    "year": 2019,
    "average_price": 420000
  },
```

```
 {
    "year": 2020,
    "average_price": 450000
  }
]
```

4.9 Most Expensive Streets

Returns streets with the highest average property price in a location.

Endpoint

GET /analytics/top-expensive-streets

Parameters

Parameter	Type	Description
location	string	Town or city
property_type	string	Property category
min_sales	integer	Minimum number of transactions
limit	integer	Maximum results returned

Example Request

GET /analytics/top-expensive-streets?location=London&min_sales=5

Example Response


```
[
  {
    "street": "Park Lane",
    "town": "London",
    "sales": 12,
    "average_price": 2450000
  },
  {
    "street": "Kensington High Street",
    "town": "London",
    "sales": 8,
    "average_price": 1890000
  }
]
```

---

5. Error Handling

The API returns standard HTTP status codes.

Code	Description
200	Successful request
400	Invalid request parameters
404	Resource not found
500	Internal server error

---

6. Interactive API Documentation

The API automatically generates interactive documentation using FastAPI.

Swagger interface: /docs

Alternative documentation: /redoc

These interfaces allow users to:

- view available endpoints
- test API requests
- inspect response schemas